



Database After Graduation

Week 4 Activity

Concurrency Testing with FastAPI + SQLAlchemy

By Narges Yarahmadi
Feb 2026

This activity will walk you step-by-step through setting up your database, adding a new stock column, and then running concurrency tests against **write endpoints** so you can observe how systems behave under real-world simultaneous users.

If you completed the Week 2 activity, great. If not, no worries. Week 2 was a foundation that makes Week 3 easier to understand, but you can still complete everything here by following the steps below carefully.

What you will learn

By the end of this activity, you will be able to:

- Add a new field (stock) to an existing database table.
- Connect FastAPI + SQLAlchemy ORM models to updated database schema.
- Implement a write endpoint (POST) that changes database state.
- Run load tests that simulate concurrent users hitting the API.
- Understand why safe transaction boundaries and row locking matter.

Step 1. Database Setup (Same as week 2 setup):

Create the database and insert initial data

1. Locate the file:
creat_db_insert_to_db.sql
2. Open it and run it in your SQL client.
This will create the database and insert initial data.

After running it successfully, your database exists and contains tables and records.

Step 2. Update the Products table to support stock

Now we will update the schema so products have a stock value.

1. Locate and open:
update_product_table.sql
2. Run it in MySQL.

This script does the following:

```
use prod_sql_week2;
```

```

ALTER TABLE products
ADD COLUMN stock INT NOT NULL DEFAULT 0;
SET SQL_SAFE_UPDATES = 0;

UPDATE products
SET stock = CASE
    WHEN RAND() < 0.3 THEN FLOOR(1 + RAND() * 10) -- 30% chance: low stock (1-10)
    WHEN RAND() < 0.6 THEN FLOOR(11 + RAND() * 40) -- 30% chance: medium stock (11-50)
    ELSE FLOOR(51 + RAND() * 450) -- 40% chance: high stock (51-500)
END;

select * from products;

```

What this script is doing

- **USE prod_sql_week2;**
Selects the database you are working with.
- **ALTER TABLE ... ADD COLUMN stock ...**
Adds a new integer column called stock to the products table.
It cannot be NULL and defaults to 0.
- **SET SQL_SAFE_UPDATES = 0;**
MySQL safe-update mode can block UPDATE queries. This disables it so the bulk update works.
- **UPDATE products SET stock = CASE ...**
Assigns random stock values to each product:
 - 30% chance: stock between 1 and 10
 - 30% chance: stock between 11 and 50
 - 40% chance: stock between 51 and 500
- **SELECT * FROM products;**
Shows you the updated table so you can confirm stock is now populated.

At this point, your database schema is ready for Week 3.

Step 3. Update ORM models and APIs

Open the Week 3 folder where the FastAPI project lives.

You will work mainly in:

- models.py
- main.py

or if you are using the week4 activity models.py and main.py skip steps 4,5,6,7,8,9. And directly go to step 10.

Step 4. Update `models.py` : Confirm Product model includes stock

Navigate to `models.py`. You should see the Product model updated to include a stock column:

```
class Product(Base):
    __tablename__ = 'products'

    product_id = Column(BigInteger, primary_key=True)
    sku = Column(String(50), nullable=False, unique=True)
    name = Column(String(255), nullable=False)
    category = Column(String(80), nullable=False)
    price = Column(DECIMAL(10,2), nullable=False)
    is_active = Column(Boolean, nullable=False, default=True)
    created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
    stock = Column(Integer, nullable=False, default=0)

    order_items = relationship("OrderItem", back_populates="product")
```

Why this matters

Your database now has a `stock` column, so the ORM must also know about it. Otherwise:

- your API won't be able to read the stock value
- your write endpoints won't be able to update stock

Step 5. Understand the request models (schemas)

In this activity, you will use these request schemas:

```
class PurchaseRequest(BaseModel):
    qty: int = Field(..., ge=1, description="Quantity to purchase (>= 1)")

class StockResetRequest(BaseModel):
    stock: int = Field(..., ge=0, description="New stock value (>= 0)")
```

What these do

- `PurchaseRequest` describes the JSON body for purchase requests, like:

```
{"qty": 1}
```

It enforces that `qty` must be at least 1.

- `StockResetRequest` describes the JSON body for resetting stock, like:

```
{"stock": 50}
```

It enforces that stock cannot be negative.

Important note: these schemas are typically placed in `main.py` (or `schemas.py`), not inside `models.py`.

Step 6. Update `main.py`: Add schemas in `main.py`

In `main.py`, add:

```
class PurchaseRequest(BaseModel):
    qty: int = Field(..., ge=1)

class StockResetRequest(BaseModel):
    stock: int = Field(..., ge=0)
```

These schemas allow FastAPI to validate JSON request bodies automatically.

Step 7. Return stock in the products listing API

In the existing `GET /products` endpoint, we now return stock too.

Add this field to the response:

```
"stock":int(r.stock)
```

So the items include:

```
{
    "product_id": r.product_id,
    "sku": r.sku,
    "name": r.name,
    "category": r.category,
    "price": float(r.price),
    "is_active": int(r.is_active),
    "stock":int(r.stock)
}
```

This helps you see stock values before and after tests.

Step 8. Add concurrency-focused endpoints: Add stock utility endpoints

These endpoints let you quickly check and reset stock.

```
@app.get("/products/{product_id}/stock")
def get_stock(product_id: int, db: Session = Depends(get_db)):
    p = db.query(Product).filter(Product.product_id == product_id).one_or_none()
    if not p:
        raise HTTPException(status_code=404, detail="product not found")
    return {"product_id": int(p.product_id), "sku": p.sku, "stock": int(p.stock)}

@app.post("/products/{product_id}/stock/reset")
def reset_stock(product_id: int, body: StockResetRequest, db: Session = Depends(get_db)):
    with db.begin():
        p = (
            db.query(Product)
            .filter(Product.product_id == product_id)
            .with_for_update()
            .one_or_none()
        )
        if not p:
            raise HTTPException(status_code=404, detail="product not found")
        p.stock = body.stock
        db.add(p)

    return {"ok": True, "product_id": int(product_id), "new_stock": int(body.stock)}
```

What to notice

- with db.begin() creates a transaction boundary.
- with_for_update() locks the row so multiple requests do not update the same product at the same time during reset.

Step 9. Add purchase endpoints (Unsafe and Safe)

Unsafe endpoint (demonstrates race conditions)

```
@app.post("/products/{product_id}/purchase-unsafe")
def purchase_unsafe(product_id: int, body: PurchaseRequest, db: Session = Depends(get_db)):
    p = db.query(Product).filter(Product.product_id == product_id).one_or_none()
    if not p:
        raise HTTPException(status_code=404, detail="product not found")
```

```

if p.stock < body.qty:
    raise HTTPException(status_code=409, detail="insufficient stock")

# optional: widen the race window so it breaks more reliably in class
time.sleep(random.uniform(0.001, 0.01))

p.stock -= body.qty
db.add(p)
db.commit()
db.refresh(p)

return {"ok": True, "product_id": int(product_id), "remaining_stock": int(p.stock)}

```

Why this is unsafe:

- Many requests can read the same stock value before any commit finishes.
- This can cause lost updates or incorrect remaining stock.

Safe endpoint (transaction + row lock)

```

@app.post("/products/{product_id}/purchase")
def purchase_safe(product_id: int, body: PurchaseRequest, db: Session = Depends(get_db)):
    try:
        with db.begin():
            p = (
                db.query(Product)
                .filter(Product.product_id == product_id)
                .with_for_update()
                .one_or_none()
            )
            if not p:
                raise HTTPException(status_code=404, detail="product not found")

            if p.stock < body.qty:
                raise HTTPException(status_code=409, detail="insufficient stock")

            p.stock -= body.qty
            db.add(p)

        return {"ok": True, "product_id": int(product_id), "remaining_stock": int(p.stock)}

    except OperationalError:
        # lock wait timeout / deadlock under high contention
        raise HTTPException(status_code=409, detail="conflict, retry")

```

```
@app.get("/products/{product_id}/stock")
def product_stock(product_id: int, db: Session = Depends(get_db)):
    p = db.query(Product).filter(Product.product_id == product_id).one_or_none()
    if not p:
        raise HTTPException(status_code=404, detail="product not found")
    return {"product_id": product_id, "stock": p.stock}
```

Why this is safe:

- Row locking prevents two updates from happening at the same time.
- The system fails safely using 409 instead of corrupting stock.

Step 10. Terminal Setup and Testing: Install a concurrency testing tool (CLI)

Install hey:

```
brew install hey
```

Confirm it works:

```
hey
```

Step 11. Start your FastAPI server

From your project directory:

```
uvicorn main:app --reload
```

Step 12. Reset stock before testing

Choose a product ID (example: product 3) and set stock to 50:

```
curl -X POST http://localhost:8000/products/3/stock/reset \
-H "Content-Type: application/json" \
-d '{"stock":50}'
```

Confirm:

```
curl http://localhost:8000/products/3/stock
```

Step 13. Run concurrency test on **UNSAFE** endpoint

1. Reset stock again:

```
curl -X POST http://localhost:8000/products/3/stock/reset \
-H "Content-Type: application/json" \
-d '{"stock":50}'
```

2. Run:

```
hey -c 100 -n 1000 -m POST -H "Content-Type: application/json" \
-d '{"qty":1}' \
http://localhost:8000/products/3/purchase-unsafe
```

3. Then check stock:

```
curl http://localhost:8000/products/3/stock
```

Observation goal:

- You may see incorrect final stock.
- This demonstrates why race conditions are dangerous.

Understanding the **UNSAFE** Concurrency Test

If the system were perfectly safe and stock started at 50, exactly 50 purchases should succeed, the final stock should be 0, and all additional requests should fail. That is the correct logical outcome.

However, this endpoint does not use `with db.begin()` or `.with_for_update()`. Because of this, multiple requests can read the same stock value at the same time, all believe inventory is still available, subtract from the same original value, and then commit overlapping updates. This creates what is known as a **lost update problem**.

After running the test, you may observe that the final stock is not zero, the stock value is incorrect, or even negative. You might also notice that the number of successful (200) responses does not match the available inventory. These are classic symptoms of a race condition.

To understand why this happens, imagine five users all reading `Stock = 50` before any transaction commits. Each calculates $50 - 1 = 49$ and writes that value back. Instead of reducing stock to 45, the database may incorrectly end up at 49. The system appears to function, but the data is wrong.

In real-world systems, this kind of unsafe concurrency could lead to overselling products, double charging customers, corrupted balances, and inconsistent financial records under load. This is why proper transaction boundaries and locking mechanisms are critical.

After running the unsafe test, compare the expected stock with the actual stock, and compare the number of successful purchases with the original inventory. This exercise shows that a system that

Step 14. Run concurrency test on **SAFE** endpoint

1. Reset the stock to a known value. Start from a controlled state again:

```
curl -X POST http://localhost:8000/products/3/stock/reset \
-H "Content-Type: application/json" \
-d '{"stock":50}'
```

2. Confirm the reset worked:

```
curl http://localhost:8000/products/3/stock
```

3. Expected result:

```
{"product_id":3,"stock":50}
```

4. Simulate heavy concurrent traffic against the SAFE endpoint: Now run the same load test, but target the **safe** endpoint:

```
hey -c 100 -n 1000 -m POST -H "Content-Type: application/json" \
-d '{"qty":1}' \
http://localhost:8000/products/3/purchase
```

What this means:

- -c 100 → 100 concurrent users
- -n 1000 → 1000 total purchase attempts
- Each request tries to buy 1 unit

5. Check the final stock

```
curl http://localhost:8000/products/3/stock
```

Expected final result:

- Stock ends at **0**
- No negative values
- No incorrect remaining stock

Understanding the **SAFE** Concurrency Test

If stock starts at 50 and each request attempts to purchase 1 unit, the safe endpoint should produce a very predictable result. Exactly 50 requests should return status 200 (success), and the remaining 950 requests should return status 409 (insufficient stock). This is the correct and expected outcome under heavy concurrency.

This behavior is a good sign. It means the system did not oversell inventory, the database remained consistent, and once stock reached zero, additional requests failed in a controlled and safe way. Most importantly, concurrency did not produce lost updates or corrupted values.

The reason this works is because `.with_for_update()` causes the database to treat that product row as a single shared resource. When one request begins updating the row, it locks it. That request checks stock, subtracts the quantity, and commits the transaction. Only after the commit completes does the next waiting request acquire the lock and repeat the process. Even if 100 users hit the endpoint at the same time, the updates occur sequentially

Key takeaway

The SAFE endpoint proves an important industry rule:

Under high concurrency, correctness comes from transaction boundaries and locking, not from “hoping the timing works out.”